

DEATH IS NOT GAME OVER

THE CURIQUINGUE TAMAGOTCHI CIRCUIT — ASSEMBLY MANUAL

SEE IT RUNNING

These clips were recorded running the program that was current on the Arduino at that time:

- youtube.com/shorts/MdQM2oSGQp8
- youtube.com/shorts/F8pmUEmmahM

The code in this manual — saved in the project's GitHub repository as **Demo Code** — is a functional demo that aspires to the full experience shown in those videos and described in the Game Flow below. It is not yet the complete, eight-step cycle end to end. See "About This Demo Code" just below for exactly what it does and doesn't do, and why.

WHAT THIS DEMO IS

The Curiquingue Tamagotchi Circuit is a hand-built Arduino arcade where a pixel-art Andean raptor tries to survive the increasingly polluted Chibunga River, flowing down from Chimborazo's glaciers. In Kichwa tradition, its rivers and springs are inhabited by elemental beings that cannot exist where water is contaminated — in strictly scientific terms, contaminated water cannot sustain life. The curiquingue, understood in Andean tradition as a messenger between earth and sky, becomes here a real indicator of environmental health, surpassing its folkloric value.

It descends from its páramo home to dance in Riobamba's Pase del Niño, carrying that journey into the river below. Struck too many times, it passes into Uku Pacha, searching a maze for clean water — then rises into Hanan Pacha, whole again, its light falling back as rain.

ABOUT THIS DEMO CODE — READ THIS FIRST

For technology-agnostic readers: the Arduino Uno is a small, inexpensive microcontroller board with very little onboard memory — only 2KB of RAM, shared between everything the program needs to keep track of while running. That is not enough space to hold the entire eight-step Game Flow described above as one continuous program.

Because of that, the Demo Code in this manual runs only part of the full cycle: the mountain descent scene (Kay Pacha), the maze (Uku Pacha), and a handful of the transition animations between them — not the complete loop through Hanan Pacha and back. It is a first, working approach, not the finished game.

The next step is migrating this same design to an ESP32 microcontroller, which has roughly 250 times more RAM (520KB) — enough to run the entire Game Flow as a single, uninterrupted program, exactly as described above and shown in the videos.

GAME FLOW

1. Splash — "Curiquingue Tamagotchi" fades in, line by line.
 2. Instructions.
 3. Kay Pacha, intro — the curiquingue walks in place.
 4. Kay Pacha, the descent — descending 3000 m.a.s.l. from the Chimborazo volcano basin, down into the Riobamba valley, to dance at the Pase del Niño. Hazards begin; three hits, or the full distance survived, trigger the threshold.
 5. Uku Pacha — a red strobe entry, then the maze, centered on screen, in search of the pogyo (Kichwa for water spring).
 6. Hanan Pacha, ascend — the curiquingue rises, whole again.
 7. Hanan Pacha, the Andean glyph rain — chakana, spiral, inti, and the rest fall before the cycle turns over.
 8. "Death is not game over" — held, then the loop returns to step 3.
-

INTRODUCTION TO CURIQUINGUE UMBRAL

Ever wondered how a small screen and a handful of lights can tell a whole story? This project introduces you to Curiquingue Umbral — a hand-built Arduino arcade where a pixel-art Andean raptor descends from the mountains, faces a polluted river, and passes through a threshold instead of simply "losing." In this build, we're using an Arduino Uno, a small OLED display, an analog joystick, and a strip of addressable NeoPixel LEDs — all parts that are affordable and beginner-friendly, even if the story they tell is not simple at all.

KEY COMPONENTS

1. Arduino Uno — controls everything: reads the joystick, draws the screen, and drives the lights.
2. Analog Joystick — lets the player move the curiquingue and jump.
3. SSD1306 OLED Display (128×64) — shows the game itself.
4. 22-LED NeoPixel Strip — lights that change color depending on what is happening in the game.
5. Jumper wires — for making all the connections.
6. Breadboard — to hold everything together without soldering, while prototyping.

OPERATION

In this circuit:

- The joystick works as a movement controller, the same way a potentiometer does:
 - Moving it left/right changes a voltage on the X axis (0–5V)
 - Moving it up/down changes a voltage on the Y axis (0–5V)
 - Arduino reads both as values from 0–1023
- The OLED shows whichever scene the game is currently in — the descent, the maze, or one of the threshold moments.
- The NeoPixel strip is updated on every loop, and always reflects the current game state — it is never just decorative lighting.
- Arduino redraws the whole screen every loop — there are no saved frames, everything is calculated live.

REQUIRED MATERIALS

1. Arduino Uno
 - USB cable / USB-C adapter, if needed

2. Analog Joystick module
3. SSD1306 OLED Display, 128×64, I2C
4. NeoPixel Strip, 22 LEDs (WS2812B)
5. Jumper wires
6. Breadboard

COMPONENTS

Electronic Component	What it does
Arduino Uno	The board that runs the whole game — reads the joystick, draws the screen, drives the lights.
Analog Joystick module	Two potentiometers (X/Y) plus a click button. Moves the curiquestue and triggers jumps.
SSD1306 OLED display (128×64, I2C)	The screen. Shows the descent, the maze, and every threshold scene.
22-LED NeoPixel strip (WS2812B)	Addressable RGB LEDs. Never just decoration — always tied to the current game phase.

TOOLS NEEDED

- Wire strippers
- Computer with Arduino IDE installed

STEP-BY-STEP ASSEMBLY GUIDE

SAFETY FIRST

- Always disconnect power before making connections
- Avoid touching electronic components directly
- Work on a clean, non-metallic surface

ASSEMBLY STEPS

Part 1: Initial Setup

1. Place your Arduino on the left side of the breadboard
2. Make sure your workspace is clear and well-lit

Part 2: Joystick Connection

1. Identify the joystick pins: GND, +5V, VRX, VRY, SW
2. Connect the joystick wires:

Connect GND from the joystick to Arduino's GND pin

Connect +5V from the joystick to Arduino's 5V pin

Connect VRX from the joystick to Arduino's A0 pin

Connect VRY from the joystick to Arduino's A1 pin

Note: the click button (SW) is wired but not used for jumping — jumping is read from the Y axis instead, moving the stick up. This was a deliberate change after testing the SW pin's behavior on this board.

Part 3: OLED Display Connection

1. Identify the OLED pins: VCC, GND, SCL, SDA
2. Connect the OLED wires:

Connect VCC from the OLED to Arduino's 5V pin

Connect GND from the OLED to Arduino's GND pin

Connect SCL from the OLED to Arduino's A5 pin

Connect SDA from the OLED to Arduino's A4 pin

Part 4: NeoPixel Strip Connection

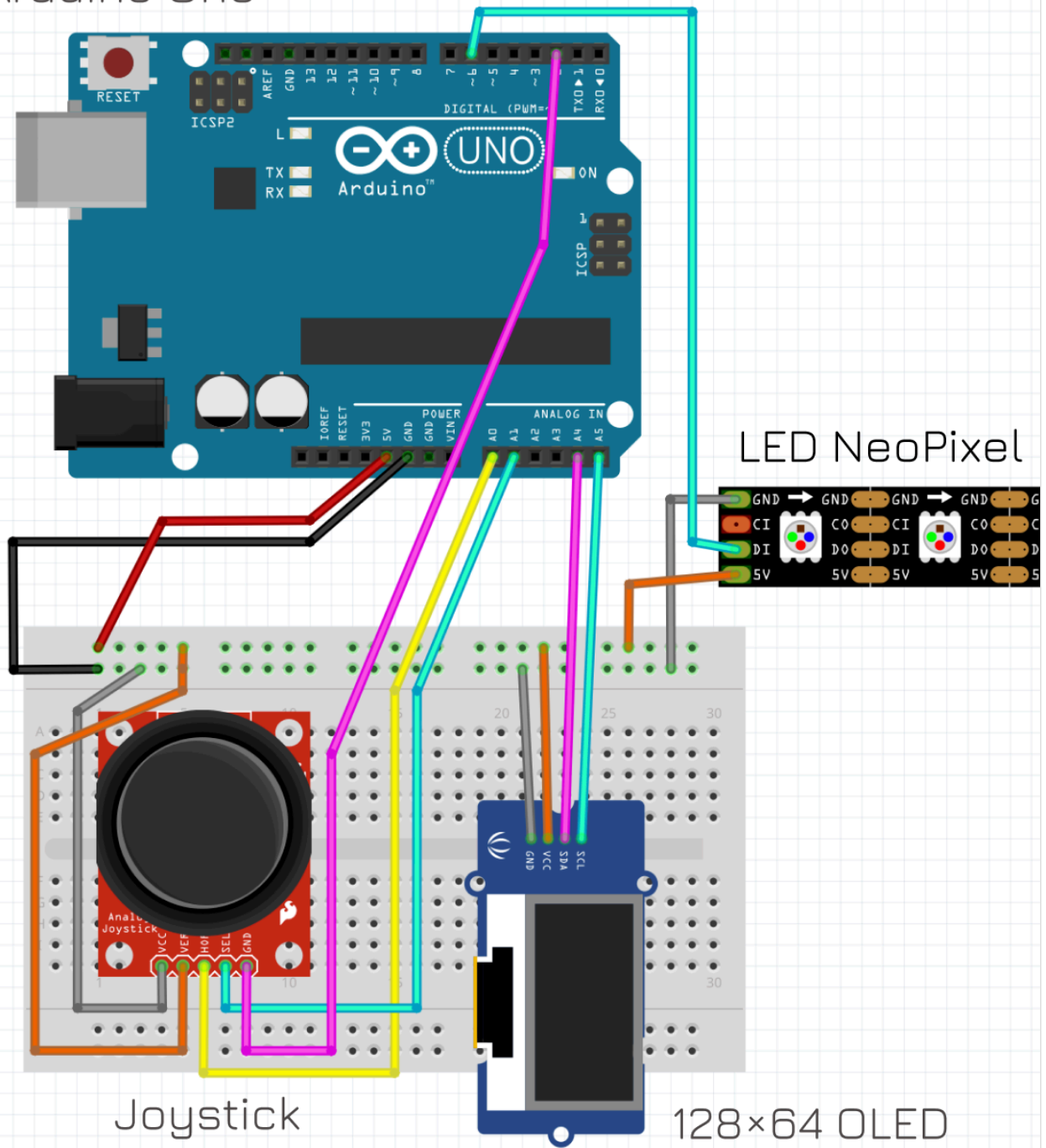
1. Identify the NeoPixel pins: GND, DIN, 5V
2. Connect the NeoPixel wires:

Connect GND from the NeoPixel strip to Arduino's GND pin

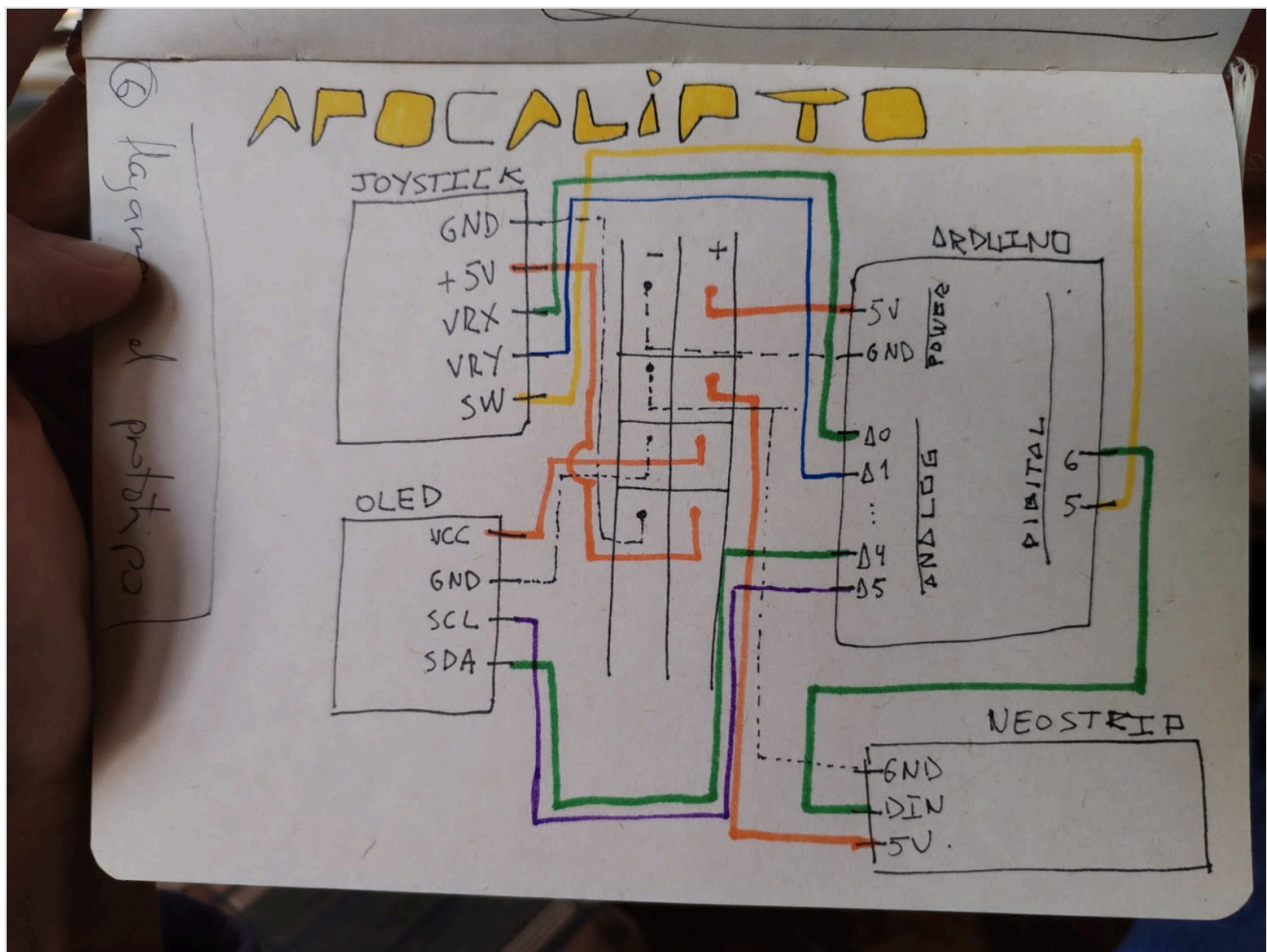
Connect DIN from the NeoPixel strip to Arduino's Pin 6

Connect 5V from the NeoPixel strip to Arduino's 5V pin

Arduino Uno



Full wiring diagram — joystick, OLED, and NeoPixel strip on the Arduino Uno.



Juan's original hand-drawn schematic — sketched before wiring.

SOFTWARE UPLOAD

1. Connect the Arduino to your computer via USB
2. Open the Arduino IDE
3. Install the required libraries, from Tools > Manage Libraries:
 - Adafruit NeoPixel
 - Adafruit GFX Library
 - Adafruit SSD1306
4. Open Demo Code (the .ino file) from the project repository
5. Select the correct board:
 - Tools > Board > Arduino Uno
6. Select the correct port:
 - Tools > Port > (select the new COM port)
7. Click the Upload button (arrow icon)

THE CODE

This is the complete Demo Code, exactly as it's saved and flashed to the physical Arduino Uno shown in the videos above — the full six-state design in one sketch. See "About This Demo Code" earlier in this manual for

what the Uno's memory limits mean for how much of this cycle runs reliably in practice, and why the ESP32 is the next step.

Note on this section: the code below sits inside a click-to-expand tab for on-screen reading. If you're saving this page as a PDF, make sure the tab is expanded (click it open) before printing/exporting — a PDF is a static snapshot, so a collapsed tab will print collapsed, and the code won't appear at all in the saved file.

▼ CLICK TO EXPAND / COLLAPSE THE FULL DEMO CODE (CURIQUINGUE_UMBRALE.INO)

```
/*
 * =====
 * CURIQUINGUE: UMBRAL — descent + maze (Arduino Uno version)
 * GLITCH 2026: Apocalypse Technologies — Juan Fernando Yanqui Rivera
 * =====
 *
 * Circuit: original joystick body (X/Y/SW + 22-NeoPixel strip)
 * with Elemental's 128x64 OLED screen mounted in place of the
 * damaged 128x32 screen.
 *
 * Hardware:
 *   - Arduino Uno
 *   - Analog joystick -> X: A0 (move / maze), Y: A1 (maze), SW: D2 (jump)
 *   - NeoPixel strip (22) -> D6
 *   - 128x64 SSD1306-compatible OLED, I2C -> A4 (SDA), A5 (SCL)
 *
 * GAMEPLAY
 *   Kay Pacha (descent): the curiquingue comes down from Chimborazo toward
 *   the city — joystick X moves, the SW button jumps over hazards on the
 *   ground (toxic / gas mask / radioactive). The background mountains are
 *   calculated live, nothing is saved, and pollution increases with
 *   distance traveled, not with time. 3 hits = collapse.
 *
 *   Maze (aerial): once the run's distance is reached, the view switches
 *   to a top-down plan — the curiquingue appears as a small triangle,
 *   seen from the sky. The maze is hand-designed and stored in flash
 *   (not algorithmically generated, to avoid risking the Uno's stack
 *   memory). Finding the inti glyph (the clean water source) triggers the
 *   same revelation as collapsing — two paths, one threshold.
 *
 *   Uku Pacha / Hanan Pacha / Hold / Restart: unchanged from the
 *   previous version — the strobing whiteout, the ascent, and the
 *   Andean restart rain all work exactly the same.
 *
 * HOOK FOR THE REAL TDS SENSOR (Elemental)
 *   getSpawnInterval() now receives the progress fraction (0-1) instead of
 *   elapsed time. To connect the real TDS sensor, just feed that fraction
 *   with a normalized sensor reading.
 *
 * NOTE ON SCALE
 *   The descent and the maze are deliberately simplified compared to the
 *   browser version (p5.js): no saved random levels, no real-time maze
 *   generation. The Uno has only 2KB of total RAM, and the screen alone
 *   already uses 1KB of that. Once we move to an ESP32 (520KB of RAM),
 *   we get the full procedural generation back with none of these limits.
 * =====
 */

#include <Adafruit_NeoPixel.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Fonts/Org_01.h> // small, irregular-stroke font — used for the pacha labels
```



```
// -----
//  HARDWARE CONFIG
//  -----
#define LED_PIN 6
#define LED_COUNT 22

#define JOY_X A0
#define JOY_Y A1
#define JOY_SW 2

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
#define SCREEN_ADDRESS 0x3C

Adafruit_NeoPixel strip(LED_COUNT, LED_PIN, NEO_GRB + NEO_KHZ800);
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

// -----
//  CURIQUINGUE SPRITE - ALERT state, copied as-is from Elemental.
//  2 frames: the head turns right <-> left (32x32, 128 bytes each).
//  -----
const unsigned char PROGMEM curi_alert_a[] = {
  0b00000000, 0b00000000, 0b00000000, 0b00000000,
  0b00000000, 0b00000001, 0b10000000, 0b00000000,
  0b00000000, 0b00000011, 0b11000000, 0b00000000,
  0b00000000, 0b00000001, 0b10000000, 0b00000000,
  0b00000000, 0b00000001, 0b10000000, 0b00000000,
  0b00000000, 0b00000011, 0b11000000, 0b00000000,
  0b00000000, 0b00000111, 0b11100000, 0b00000000,
  0b00000000, 0b00001111, 0b11110000, 0b00000000,
  0b00000000, 0b00011111, 0b11111000, 0b00000000,
  0b00000000, 0b00111111, 0b00111100, 0b00000000,
  0b00000000, 0b01111111, 0b11111111, 0b00000000,
  0b00000000, 0b01111111, 0b11111111, 0b11000000,
  0b00000000, 0b00111111, 0b11111110, 0b00000000,
  0b00000000, 0b00011111, 0b11111000, 0b00000000,
  0b00000000, 0b00001111, 0b11110000, 0b00000000,
  0b00000000, 0b11111111, 0b11111111, 0b00000000,
  0b00000001, 0b11111000, 0b00001111, 0b10000000,
  0b00000011, 0b11100000, 0b00000111, 0b11000000,
  0b00000111, 0b11000000, 0b00000011, 0b11100000,
  0b00000111, 0b01000000, 0b00000010, 0b11100000,
  0b00000111, 0b01000000, 0b00000010, 0b11100000,
  0b00000111, 0b01100000, 0b00000110, 0b11100000,
  0b00000111, 0b01110000, 0b00001110, 0b11100000,
  0b00000011, 0b00111111, 0b11111100, 0b11000000,
  0b00000001, 0b00011111, 0b11111000, 0b10000000,
  0b00000000, 0b00001111, 0b11110000, 0b00000000,
  0b00000000, 0b00000111, 0b11100000, 0b00000000,
  0b00000000, 0b00000011, 0b00000000, 0b00000000,
  0b00000000, 0b00000011, 0b01100000, 0b00000000,
  0b00000000, 0b00000011, 0b01100000, 0b00000000,
  0b00000000, 0b00000111, 0b11110000, 0b00000000,
};

const unsigned char PROGMEM curi_alert_b[] = {
  0b00000000, 0b00000000, 0b00000000, 0b00000000,
  0b00000000, 0b00000001, 0b10000000, 0b00000000,
  0b00000000, 0b00000011, 0b11000000, 0b00000000,
  0b00000000, 0b00000001, 0b10000000, 0b00000000,
  0b00000000, 0b00000001, 0b10000000, 0b00000000,
  0b00000000, 0b00000011, 0b11000000, 0b00000000,
  0b00000000, 0b00000111, 0b11110000, 0b00000000,
  0b00000000, 0b00011111, 0b11111000, 0b00000000,
  0b00000000, 0b00111111, 0b00111100, 0b00000000,
  0b00000001, 0b00011111, 0b11111000, 0b10000000,
  0b00000000, 0b00001111, 0b11110000, 0b00000000,
  0b00000000, 0b00000111, 0b11100000, 0b00000000,
  0b00000000, 0b00000011, 0b00000000, 0b00000000,
  0b00000000, 0b00000011, 0b01100000, 0b00000000,
  0b00000000, 0b00000011, 0b01100000, 0b00000000,
  0b00000000, 0b00000111, 0b11110000, 0b00000000,
  0b00000000, 0b00011111, 0b11111000, 0b00000000,
  0b00000000, 0b00111111, 0b00111100, 0b00000000,
};
```



```

0b00000000, 0b11111111, 0b11111110, 0b00000000,
0b00000011, 0b11111111, 0b11111110, 0b00000000,
0b00000000, 0b01111111, 0b11111100, 0b00000000,
0b00000000, 0b00011111, 0b11111000, 0b00000000,
0b00000000, 0b00001111, 0b11110000, 0b00000000,
0b00000000, 0b11111111, 0b11111111, 0b00000000,
0b00000001, 0b11111000, 0b00001111, 0b10000000,
0b00000011, 0b11000000, 0b00000111, 0b11000000,
0b00000111, 0b11000000, 0b00000011, 0b11100000,
0b00000111, 0b01000000, 0b00000010, 0b11100000,
0b00000111, 0b01000000, 0b00000010, 0b11100000,
0b00000111, 0b01100000, 0b00000110, 0b11100000,
0b00000111, 0b01110000, 0b00001110, 0b11100000,
0b00000111, 0b01111000, 0b00011110, 0b11100000,
0b00000011, 0b00111111, 0b11111100, 0b11000000,
0b00000001, 0b00011111, 0b11111000, 0b10000000,
0b00000000, 0b00001111, 0b11110000, 0b00000000,
0b00000000, 0b00000111, 0b11100000, 0b00000000,
0b00000000, 0b00000011, 0b00000000, 0b00000000,
0b00000000, 0b00000011, 0b01100000, 0b00000000,
0b00000000, 0b00000011, 0b01100000, 0b00000000,
0b00000000, 0b00000111, 0b11100000, 0b00000000,
};

// -----
// DESCENT LAYOUT
// -----

#define GROUND_Y 54
#define BIRD_X 20
#define BIRD_SMALL 16
#define HAZARD_RADIUS 5
#define MAX_HAZARDS 4
#define MAZE_TRIGGER_DISTANCE 420.0f
#define MOUNTAIN_DELAY_MS 6000UL

#define GRAVITY 0.30f
#define JUMP_V -4.2f

// -----
// PACE / DIFFICULTY (by distance traveled - replaceable with TDS data later)
// -----
#define SPAWN_MAX 1000UL
#define SPAWN_MIN 380UL

// -----
// LIVES
// -----
#define INVULN_MS 900UL
#define HIT_PULSE_MS 260UL

// -----
// THRESHOLD PHASE DURATIONS
// -----
#define FLASH_DURATION 3400UL
#define STROBE_GROW_FRAC 0.35
#define STROBE_MIN_INTERVAL 25UL
#define STROBE_MAX_INTERVAL 90UL
#define GLITCH_FLASH_CHANCE 18
#define ASCEND_DURATION 3200UL
#define HOLD_DURATION 3500UL
#define RESTART_DURATION 30000UL

// -----
// GAME STATE
// -----
enum GameState { KAY_PACHA, MAZE_PACHA, UKU_PACHA, HANAN_PACHA, HOLD_PACHA, RESTART_TRANSITION };
GameState state = KAY_PACHA;
unsigned long stateStart = 0;

```

```

float worldX = 0;
float birdY = GROUND_Y - BIRD_SMALL;
float vy = 0;
bool onGround = true;
int lives = 3;
unsigned int score = 0;
bool joyUpWasActive = false;

unsigned long invulnerableUntil = 0;
unsigned long hitPulseUntil = 0;

struct Hazard {
    bool active;
    float x;          // position in the world, not on screen
    int type;
    bool scored;
};
Hazard hazards[MAX_HAZARDS];
unsigned long lastSpawn = 0;

unsigned long lastStripUpdate = 0;

bool strobeOn = false;
unsigned long lastStrobeToggle = 0;

// -----
// MAZE - hand-designed, stored in flash (16x8 cells, 8px each).
// A single winding path from (0,0) to the goal (14,6), with a few
// vertical corridors. Not algorithmically generated - on the Uno, a real
// generator's recursion risks the little stack memory available.
// -----
const uint16_t PROGMEM mazeRows[8] = {
    0b00000010001000111,
    0b1111010101010111,
    0b1100010101010111,
    0b1101110101010111,
    0b1101110101010111,
    0b1101110101010111,
    0b1101110101010111,
    0b1100001000100001,
    0b1111111111111111,
};
#define MAZE_GOAL_X 14
#define MAZE_GOAL_Y 6
#define MAZE_CELL 8

int mGridX = 0, mGridY = 0;
int mFacingDX = 1, mFacingDY = 0;
unsigned long lastMazeMove = 0;
#define MAZE_MOVE_COOLDOWN 150UL

// -----
// RESTART RAIN - vertical matrix with Andean motifs
// -----
#define NUM_COLUMNS 11
struct MatrixColumn {
    float y;
    float speed;
    int glyph;
};
MatrixColumn columns[NUM_COLUMNS];

// =====
// SETUP
// =====
void setup() {
    Serial.begin(9600);
    pinMode(JOY_SW, INPUT_PULLUP);

```

```

strip.begin();
strip.show();
strip.setBrightness(60);

if (!display.begin(SSD1306_SWITCHCAPVCC, SCREEN_ADDRESS)) {
    Serial.println(F("OLED failed"));
}
display.clearDisplay();
display.setTextSize(1);
display.setTextColor(SSD1306_WHITE);
display.setCursor(4, 16);
display.println(F("Curiquingue: Umbral"));
display.setCursor(4, 32);
display.println(F("X moves, up jumps"));
display.setCursor(4, 44);
display.println(F("maze: X + Y"));
display.display();
delay(1800);

startKayPacha();
}

// =====
// MAIN LOOP
// =====
void loop() {
    switch (state) {
        case KAY_PACHA:      loopKayPacha();      break;
        case MAZE_PACHA:     loopMazePacha();      break;
        case UKU_PACHA:      loopUkuPacha();       break;
        case HANAN_PACHA:    loopHananPacha();     break;
        case HOLD_PACHA:     loopHoldPacha();      break;
        case RESTART_TRANSITION: loopRestartTransition(); break;
    }
}

// =====
// PHASE 1 - KAY PACHA (descent from Chimborazo, platformer)
// =====
void startKayPacha() {
    state = KAY_PACHA;
    stateStart = millis();
    worldX = 0;
    birdY = GROUND_Y - BIRD_SMALL;
    vy = 0;
    onGround = true;
    lives = 3;
    score = 0;
    invulnerableUntil = 0;
    hitPulseUntil = 0;
    for (int i = 0; i < MAX_HAZARDS; i++) hazards[i].active = false;
    lastSpawn = millis();
}

void loopKayPacha() {
    readInput();
    updateHazards();
    checkCollision();
    drawKayPacha();
    updateStripKay();

    if (lives <= 0) {
        state = UKU_PACHA;
        stateStart = millis();
        for (int i = 0; i < MAX_HAZARDS; i++) hazards[i].active = false;
    } else if (worldX >= MAZE_TRIGGER_DISTANCE) {
        state = MAZE_PACHA;
    }
}

```

```

        stateStart = millis();
        resetMazePlayer();
    }
}

// ---- input: X moves, pushing the joystick up jumps -----
void readInput() {
    int x = analogRead(JOY_X);
    int y = analogRead(JOY_Y);
    float moveSpeed = 1.6;
    if (x < 400)        worldX = max(0.0f, worldX - moveSpeed * 0.4f); // step back, slower
    else if (x > 600) worldX += moveSpeed;

    bool upNow = (y < 400);
    if (upNow && !joyUpWasActive && onGround) {
        vy = JUMP_V;
        onGround = false;
    }
    joyUpWasActive = upNow;

    vy += GRAVITY;
    birdY += vy;
    if (birdY >= GROUND_Y - BIRD_SMALL) {
        birdY = GROUND_Y - BIRD_SMALL;
        vy = 0;
        onGround = true;
    }
}

// ---- difficulty: replace `progress` with a real TDS reading later -----
unsigned long getSpawnInterval(float progress) {
    progress = constrain(progress, 0.0, 1.0);
    return map((long)(progress * 1000), 0, 1000, SPAWN_MAX, SPAWN_MIN);
}

// ---- hazard spawning, static in the world (the camera moves, not them) -
void spawnHazard() {
    for (int i = 0; i < MAX_HAZARDS; i++) {
        if (!hazards[i].active) {
            hazards[i].active = true;
            hazards[i].x = worldX + SCREEN_WIDTH + random(10, 40);
            hazards[i].type = random(0, 3);
            hazards[i].scored = false;
            return;
        }
    }
}

void updateHazards() {
    unsigned long now = millis();
    float progress = worldX / MAZE_TRIGGER_DISTANCE;

    if (now - lastSpawn > getSpawnInterval(progress)) {
        spawnHazard();
        lastSpawn = now;
    }

    for (int i = 0; i < MAX_HAZARDS; i++) {
        if (!hazards[i].active) continue;
        if (!hazards[i].scored && hazards[i].x < worldX - 10) {
            hazards[i].scored = true;
            score++;
        }
        if (hazards[i].x < worldX - 60) hazards[i].active = false;
    }
}

// ---- collision: only while on the ground (didn't jump in time) -----

```

```

void checkCollision() {
    if (!onGround) return;
    if (millis() < invulnerableUntil) return;

    for (int i = 0; i < MAX_HAZARDS; i++) {
        if (!hazards[i].active) continue;
        if (fabs(worldX - hazards[i].x) < 8) {
            registerHit();
            hazards[i].active = false;
            break;
        }
    }
}

void registerHit() {
    lives--;
    invulnerableUntil = millis() + INVULN_MS;
    hitPulseUntil = millis() + HIT_PULSE_MS;
}

// ---- curiquingue at half size (16x16), for the platformer -----
void drawBirdSmall(int x, int y) {
    static unsigned long lastFrameSwap = 0;
    static bool frameA = true;
    if (millis() - lastFrameSwap > 400) { frameA = !frameA; lastFrameSwap = millis(); }
    const unsigned char* bmp = frameA ? curi_alert_a : curi_alert_b;

    for (int oy = 0; oy < BIRD_SMALL; oy++) {
        for (int ox = 0; ox < BIRD_SMALL; ox++) {
            int sy = oy * 2, sx = ox * 2;
            int byteIndex = sy * 4 + (sx / 8);
            unsigned char b = pgm_read_byte(&bmp[byteIndex]);
            if (b & (0x80 >> (sx % 8))) display.drawPixel(x + ox, y + oy, SSD1306_WHITE);
        }
    }
}

// ---- curiquingue at full size (32x32), for Hanan Pacha / resolution --
void drawBird(int x, int y) {
    static unsigned long lastFrameSwap = 0;
    static bool frameA = true;
    if (millis() - lastFrameSwap > 400) { frameA = !frameA; lastFrameSwap = millis(); }
    const unsigned char* bmp = frameA ? curi_alert_a : curi_alert_b;
    display.drawBitmap(x, y, bmp, 32, 32, SSD1306_WHITE);
}

// ---- hazard drawing -----
void drawHazard(int cx, int cy, int type) {
    switch (type) {
        case 0:
            display.fillCircle(cx, cy, HAZARD_RADIUS, SSD1306_WHITE);
            display.fillCircle(cx - 2, cy - 1, 1, SSD1306_BLACK);
            display.fillCircle(cx + 2, cy - 1, 1, SSD1306_BLACK);
            break;
        case 1:
            display.drawCircle(cx, cy, HAZARD_RADIUS, SSD1306_WHITE);
            display.fillCircle(cx - 2, cy, 1, SSD1306_WHITE);
            display.fillCircle(cx + 2, cy, 1, SSD1306_WHITE);
            display.fillRect(cx - 1, cy + 3, 2, 2, SSD1306_WHITE);
            break;
        case 2:
            display.fillCircle(cx, cy, HAZARD_RADIUS, SSD1306_WHITE);
            display.fillCircle(cx, cy, 1, SSD1306_BLACK);
            display.drawLine(cx, cy, cx, cy - HAZARD_RADIUS, SSD1306_BLACK);
            display.drawLine(cx, cy, cx - HAZARD_RADIUS, cy + 3, SSD1306_BLACK);
            display.drawLine(cx, cy, cx + HAZARD_RADIUS, cy + 3, SSD1306_BLACK);
            break;
    }
}

```

```

}

// ---- pacha label: Org_01 font (more organic), returns to the classic font
// ---- for the rest of the UI (HUD, numbers) right after -----
void drawPachaLabel(int16_t x, int16_t yTop, const __FlashStringHelper *text) {
    display.setFont(&Org_01);
    display.setCursor(x, yTop + 6);    // Org_01 anchors on the baseline, not the top-left corner
    display.print(text);
    display.setFont(NULL);
}

// ---- helper: safe modulo (never negative), for the background scroll -----
long wrapMod(long v, long m) {
    long r = v % m;
    if (r < 0) r += m;
    return r;
}

// ---- full screen drawing: mountains -> city, live -----
void drawKayPacha() {
    display.clearDisplay();

    float progress = constrain(worldX / MAZE_TRIGGER_DISTANCE, 0.0, 1.0);
    float camX = max(0.0f, worldX - 26);

    // pollution: noise dots that grow with distance traveled
    int noiseDots = (int)(progress * 10);
    for (int i = 0; i < noiseDots; i++) {
        display.drawPixel(random(SCREEN_WIDTH), random(30), SSD1306_WHITE);
    }

    // Chimborazo -> city skyline, calculated live (nothing is saved) -
    // delayed 6s at the start of the phase, so the bird is seen alone
    // against a clean background before the landscape appears.
    if (millis() - stateStart > MOUNTAIN_DELAY_MS) {
        int peakSpan = SCREEN_WIDTH / 4;
        int scrollOff = (int)wrapMod((long)(camX * 0.3), peakSpan);
        for (int i = 0; i <= 4; i++) {
            int wx = i * peakSpan - scrollOff;
            int h = 8 + (i % 2) * 5 - (int)(progress * 5);
            display.fillTriangle(wx - 12, GROUND_Y - 2, wx, GROUND_Y - 2 - h, wx + 12, GROUND_Y - 2, SS
        }
        if (progress > 0.4) {
            int stackOff = (int)wrapMod((long)(camX * 0.5), 90);
            for (int i = 0; i < 2; i++) {
                int sx = i * 70 - stackOff;
                display.fillRect(sx, GROUND_Y - 18, 3, 10, SSD1306_WHITE);
            }
        }
    }

    // ground
    display.fillRect(0, GROUND_Y, SCREEN_WIDTH, SCREEN_HEIGHT - GROUND_Y, SSD1306_WHITE);

    // hazards
    for (int i = 0; i < MAX_HAZARDS; i++) {
        if (!hazards[i].active) continue;
        float hx = hazards[i].x - camX;
        if (hx < -8 || hx > SCREEN_WIDTH + 8) continue;
        drawHazard((int)hx, GROUND_Y - 6, hazards[i].type);
    }

    // bird, with a brief blink while invulnerable after a hit
    bool blinkOff = (millis() < invulnerableUntil) && ((millis() / 80) % 2 == 0);
    if (!blinkOff) drawBirdSmall(BIRD_X, (int)birdY);

    display.setTextSize(1); display.setTextColor(SSD1306_WHITE);
    display.setCursor(0, 0); display.print(F("V:")); display.print(lives);

```

```

    display.setCursor(90, 0); display.print(F("P:")); display.print(score);

    display.display();
}

// ---- NeoPixel: lives (LEDs 0-2) + ambient ramp (LEDs 3-21) -----
void updateStripKay() {
    if (millis() - lastStripUpdate < 40) return;
    lastStripUpdate = millis();

    float frac = constrain(worldX / MAZE_TRIGGER_DISTANCE, 0.0, 1.0);

    for (int i = 0; i < 3; i++) {
        strip.setPixelColor(i, i < lives ? strip.Color(255, 255, 255) : strip.Color(0, 0, 0));
    }

    if (millis() < hitPulseUntil) {
        for (int i = 3; i < LED_COUNT; i++) strip.setPixelColor(i, strip.Color(255, 0, 0));
    } else {
        ambientChase(frac);
    }

    strip.show();
}

void ambientChase(float frac) {
    static unsigned long lastStep = 0;
    static int pos = 0;

    int r, g, b;
    if (frac < 0.5) {
        float k = frac / 0.5;
        r = (int)(k * 255); g = (int)(180 - k * 30); b = 0;
    } else {
        float k = (frac - 0.5) / 0.5;
        r = 255; g = (int)(150 - k * 150); b = 0;
    }

    for (int i = 3; i < LED_COUNT; i++) strip.setPixelColor(i, strip.Color(0, 0, 0));

    int span = LED_COUNT - 3;
    for (int i = 0; i < 4; i++) {
        int p = 3 + ((pos + i) % span);
        float fade = 1.0 - (i * 0.22);
        strip.setPixelColor(p, strip.Color((int)(r * fade), (int)(g * fade), (int)(b * fade)));
    }

    if (millis() - lastStep > 60) {
        pos = (pos + 1) % span;
        lastStep = millis();
    }
}

// =====
// PHASE 2 - AERIAL MAZE (the curiquingue seen from the sky)
// =====
bool isWallAt(int x, int y) {
    if (x < 0 || x >= 16 || y < 0 || y >= 8) return true;
    uint16_t row = pgm_read_word(&mazeRows[y]);
    return (row >> (15 - x)) & 1;
}

void resetMazePlayer() {
    mGridX = 0; mGridY = 0;
    mFacingDX = 1; mFacingDY = 0;
    lastMazeMove = 0;
}

```



```

void loopMazePacha() {
    updateMaze();
    drawMazePacha();
    stripMatrixCascade(); // the same green cascade from the restart rain, as ambient light
}

void updateMaze() {
    if (millis() - lastMazeMove < MAZE_MOVE_COOLDOWN) return;

    int x = analogRead(JOY_X), y = analogRead(JOY_Y);
    int dx = 0, dy = 0;
    if (x < 400) dx = -2;
    else if (x > 600) dx = 2;
    else if (y < 400) dy = -2;
    else if (y > 600) dy = 2;
    if (dx == 0 && dy == 0) return;

    int mx = mGridX + dx / 2, my = mGridY + dy / 2;
    int nx = mGridX + dx, ny = mGridY + dy;
    if (nx < 0 || nx >= 16 || ny < 0 || ny >= 8) return;
    if (isWallAt(mx, my)) return;

    mGridX = nx; mGridY = ny;
    mFacingDX = dx / 2; mFacingDY = dy / 2;
    lastMazeMove = millis();

    if (mGridX == MAZE_GOAL_X && mGridY == MAZE_GOAL_Y) {
        state = UKU_PACHA;
        stateStart = millis();
    }
}

void drawMazePacha() {
    display.clearDisplay();

    for (int x = 0; x < 16; x++) {
        for (int y = 0; y < 8; y++) {
            if (isWallAt(x, y)) display.fillRect(x * MAZE_CELL, y * MAZE_CELL, MAZE_CELL, MAZE_CELL, SS
        }
    }

    // goal - small inti (sun) glyph
    int gx = MAZE_GOAL_X * MAZE_CELL + MAZE_CELL / 2, gy = MAZE_GOAL_Y * MAZE_CELL + MAZE_CELL / 2;
    display.fillCircle(gx, gy, 1, SSD1306_WHITE);
    display.drawPixel(gx, gy - 3, SSD1306_WHITE); display.drawPixel(gx, gy + 3, SSD1306_WHITE);
    display.drawPixel(gx - 3, gy, SSD1306_WHITE); display.drawPixel(gx + 3, gy, SSD1306_WHITE);

    // the curiquingue, seen from the sky - small triangle oriented toward its movement
    int px = mGridX * MAZE_CELL + MAZE_CELL / 2, py = mGridY * MAZE_CELL + MAZE_CELL / 2;
    int tx1, ty1, tx2, ty2, tx3, ty3;
    if (mFacingDX > 0) { tx1 = px - 2; ty1 = py - 2; tx2 = px - 2; ty2 = py + 2; tx3 = px + 3;
    else if (mFacingDX < 0) { tx1 = px + 2; ty1 = py - 2; tx2 = px + 2; ty2 = py + 2; tx3 = px - 3;
    else if (mFacingDY > 0) { tx1 = px - 2; ty1 = py - 2; tx2 = px + 2; ty2 = py - 2; tx3 = px; ty3
    else { tx1 = px - 2; ty1 = py + 2; tx2 = px + 2; ty2 = py + 2; tx3 = px; ty3
    display.fillTriangle(tx1, ty1, tx2, ty2, tx3, ty3, SSD1306_WHITE);

    display.setTextSize(1); display.setTextColor(SSD1306_WHITE);
    display.setCursor(2, 0); display.print(F("MAZE"));
    display.display();
}

// =====
// PHASE 3 - UKU PACHA (the threshold / clear light, with strobe)
// =====
void loopUkuPacha() {
    unsigned long elapsedPhase = millis() - stateStart;
    float t = constrain((float)elapsedPhase / (float)FLASH_DURATION, 0.0, 1.0);

```

```

unsigned long interval = map((long)(t * 100), 0, 100, STROBE_MAX_INTERVAL, STROBE_MIN_INTERVAL);
if (millis() - lastStrobeToggle > interval) {
    strobeOn = !strobeOn;
    lastStrobeToggle = millis();
}

display.clearDisplay();
if (t < STROBE_GROW_FRAC) {
    int r = (int)map((long)(t * 1000), 0, (long)(STROBE_GROW_FRAC * 1000), 0, 95);
    if (strobeOn) display.fillCircle(SCREEN_WIDTH / 2, SCREEN_HEIGHT / 2, r, SSD1306_WHITE);
} else {
    if (strobeOn) display.fillRect(0, 0, SCREEN_WIDTH, SCREEN_HEIGHT, SSD1306_WHITE);
}

if (t > STROBE_GROW_FRAC && t < 0.6 && strobeOn) {
    display.setTextColor(SSD1306_BLACK);
    drawPachaLabel(24, SCREEN_HEIGHT / 2 - 4, F("UKU PACHA"));
}

display.display();

if (strobeOn) {
    bool glitch = (random(100) < GLITCH_FLASH_CHANCE);
    uint32_t col = glitch
        ? (random(2) == 0 ? strip.Color(255, 0, 90) : strip.Color(0, 200, 255))
        : strip.Color(255, 255, 255);
    for (int i = 0; i < LED_COUNT; i++) strip.setPixelColor(i, col);
} else {
    for (int i = 0; i < LED_COUNT; i++) strip.setPixelColor(i, 0);
}

strip.show();

if (elapsedPhase > FLASH_DURATION) {
    state = HANAN_PACHA;
    stateStart = millis();
}
}

// =====
// PHASE 4 - HANAN PACHA (the world above, ascent)
// =====

void drawSun(int cx, int cy) {
    display.fillCircle(cx, cy, 5, SSD1306_WHITE);
    for (int i = 0; i < 8; i++) {
        float a = i * (PI / 4.0);
        int x1 = cx + (int)(cos(a) * 8);
        int y1 = cy + (int)(sin(a) * 8);
        int x2 = cx + (int)(cos(a) * 12);
        int y2 = cy + (int)(sin(a) * 12);
        display.drawLine(x1, y1, x2, y2, SSD1306_WHITE);
    }
}

void loopHananPacha() {
    unsigned long elapsedPhase = millis() - stateStart;
    float p = constrain((float)elapsedPhase / (float)ASCEND_DURATION, 0.0, 1.0);
    drawAscendFrame(p);

    if (elapsedPhase > ASCEND_DURATION) {
        state = HOLD_PACHA;
        stateStart = millis();
    }
}

void drawAscendFrame(float p) {
    display.clearDisplay();
    drawSun(18, 10);

    float bob = sin(millis() * 0.005) * 2;
    int by = (int)(30 - p * 26 + bob);

```

```

drawBird(SCREEN_WIDTH / 2 - 16, by);

if (p > 0.35) {
    display.setTextColor(SSD1306_WHITE);
    drawPachaLabel(24, 56, F("HANAN PACHA"));
}
display.display();

ambientWarm();
}

void ambientWarm() {
    if (millis() - lastStripUpdate < 40) return;
    lastStripUpdate = millis();

    float pulse = (sin(millis() * 0.003) + 1.0) / 2.0;
    int bright = (int)(70 + pulse * 130);
    for (int i = 0; i < LED_COUNT; i++) {
        strip.setPixelColor(i, strip.Color(bright, (int)(bright * 0.75), (int)(bright * 0.2)));
    }
    strip.show();
}

// =====
// PHASE 5 - HOLD (hold Hanan Pacha, then restart the cycle)
// =====
void loopHoldPacha() {
    drawAscendFrame(1.0);

    if (millis() - stateStart > HOLD_DURATION) {
        startRestartTransition();
    }
}

// =====
// PHASE 6 - RESTART SCREEN (vertical rain, Andean motifs)
// =====
void startRestartTransition() {
    state = RESTART_TRANSITION;
    stateStart = millis();
    for (int i = 0; i < NUM_COLUMNS; i++) {
        columns[i].y = random(-90, -10);
        columns[i].speed = random(12, 32) / 10.0;
        columns[i].glyph = random(0, 10);
    }
}

void loopRestartTransition() {
    unsigned long elapsedPhase = millis() - stateStart;
    updateMatrixColumns();
    drawMatrixColumns(elapsedPhase);
    stripMatrixCascade();

    if (elapsedPhase > RESTART_DURATION) {
        startKayPacha();
    }
}

void updateMatrixColumns() {
    static unsigned long lastTick = 0;
    if (millis() - lastTick < 30) return;
    lastTick = millis();
    for (int i = 0; i < NUM_COLUMNS; i++) {
        columns[i].y += columns[i].speed;
        if (columns[i].y > SCREEN_HEIGHT + 20) {
            columns[i].y = random(-60, -10);
            columns[i].speed = random(12, 32) / 10.0;
            columns[i].glyph = random(0, 10);
        }
    }
}

```

```

    }
}

void drawMatrixColumns(unsigned long elapsedPhase) {
    display.clearDisplay();
    int spacing = SCREEN_WIDTH / NUM_COLUMNS;

    for (int i = 0; i < NUM_COLUMNS; i++) {
        int x = 3 + i * spacing;
        int y = (int)columns[i].y;
        drawAndeanGlyph(x, y, columns[i].glyph);
        display.drawPixel(x + 2, y - 9, SSD1306_WHITE);
        display.drawPixel(x + 2, y - 18, SSD1306_WHITE);
    }

    if (elapsedPhase > RESTART_DURATION / 2) {
        display.setTextColor(SSD1306_WHITE);
        drawPachaLabel(30, SCREEN_HEIGHT / 2 - 4, F("KAY PACHA"));
    }
    display.display();
}

void drawAndeanGlyph(int x, int y, int type) {
    switch (type) {
        case 0:
            display.drawFastVLine(x + 2, y, 5, SSD1306_WHITE);
            display.drawFastHLine(x, y + 2, 5, SSD1306_WHITE);
            break;
        case 1:
            display.drawLine(x + 2, y, x + 4, y + 2, SSD1306_WHITE);
            display.drawLine(x + 4, y + 2, x + 2, y + 4, SSD1306_WHITE);
            display.drawLine(x + 2, y + 4, x, y + 2, SSD1306_WHITE);
            display.drawLine(x, y + 2, x + 2, y, SSD1306_WHITE);
            display.drawPixel(x + 2, y + 2, SSD1306_WHITE);
            break;
        case 2:
            display.drawLine(x, y + 4, x + 2, y, SSD1306_WHITE);
            display.drawLine(x + 2, y, x + 4, y + 4, SSD1306_WHITE);
            display.drawLine(x + 4, y + 4, x + 6, y, SSD1306_WHITE);
            break;
        case 3:
            display.fillCircle(x + 2, y + 2, 1, SSD1306_WHITE);
            display.drawPixel(x + 2, y, SSD1306_WHITE);
            display.drawPixel(x + 2, y + 4, SSD1306_WHITE);
            display.drawPixel(x, y + 2, SSD1306_WHITE);
            display.drawPixel(x + 4, y + 2, SSD1306_WHITE);
            break;
        case 4:
            display.drawLine(x, y + 4, x + 2, y, SSD1306_WHITE);
            display.drawLine(x + 2, y, x + 4, y + 4, SSD1306_WHITE);
            display.drawFastHLine(x, y + 4, 5, SSD1306_WHITE);
            break;
        case 5:
            display.drawFastVLine(x + 2, y - 1, 7, SSD1306_WHITE);
            display.drawFastHLine(x - 1, y + 2, 7, SSD1306_WHITE);
            display.drawPixel(x, y, SSD1306_WHITE);
            display.drawPixel(x + 4, y, SSD1306_WHITE);
            display.drawPixel(x, y + 4, SSD1306_WHITE);
            display.drawPixel(x + 4, y + 4, SSD1306_WHITE);
            break;
        case 6:
            display.drawCircle(x + 2, y + 2, 3, SSD1306_WHITE);
            display.drawPixel(x + 2, y + 2, SSD1306_WHITE);
            break;
        case 7:
            display.drawLine(x, y, x + 4, y + 4, SSD1306_WHITE);
            display.drawLine(x + 4, y, x, y + 4, SSD1306_WHITE);
    }
}

```

```

        display.drawPixel(x + 2, y + 2, SSD1306_BLACK);
        break;
    case 8:
        display.drawLine(x,      y + 3, x + 1, y,      SSD1306_WHITE);
        display.drawLine(x + 1, y,      x + 3, y + 5, SSD1306_WHITE);
        display.drawLine(x + 3, y + 5, x + 5, y,      SSD1306_WHITE);
        display.drawLine(x + 5, y,      x + 6, y + 3, SSD1306_WHITE);
        break;
    case 9:
        display.drawFastHLine(x,      y,      3, SSD1306_WHITE);
        display.drawFastVLine(x + 2, y,      3, SSD1306_WHITE);
        display.drawFastHLine(x + 2, y + 2, 3, SSD1306_WHITE);
        display.drawFastVLine(x + 4, y + 2, 3, SSD1306_WHITE);
        break;
    }
}

void stripMatrixCascade() {
    static unsigned long lastStep = 0;
    static int pos = 0;
    if (millis() - lastStep < 55) return;
    lastStep = millis();

    for (int i = 0; i < LED_COUNT; i++) strip.setPixelColor(i, strip.Color(0, 0, 0));
    for (int i = 0; i < 5; i++) {
        int p = (pos + i) % LED_COUNT;
        float fade = 1.0 - (i * 0.18);
        strip.setPixelColor(p, strip.Color(0, (int)(220 * fade), (int)(60 * fade)));
    }
    strip.show();
    pos = (pos + 1) % LED_COUNT;
}

```

One small change from the version running on the physical board: the on-screen text was translated to English for this manual ("LABERINTO" → "MAZE", and the two setup instruction lines) — the actual Arduino currently displays these in Spanish. Everything else, including all logic, timing, and structure, is unchanged.

HOW THIS PROGRAM WORKS

The full design (shown in the videos and the Game Flow above) runs as a state machine: at any moment, the program is in exactly one of six states, and only draws and updates whichever state is currently active. The Demo Code in this manual implements this same structure for the mountain descent (Kay Pacha) and the maze (Uku Pacha), plus supporting transition animations — the states below describe the complete design this project is built toward, fully realized once ported to the ESP32.

1. Kay Pacha — the descent. The curiquingue walks toward the city, jumping over hazards.
2. Maze Pacha — the aerial maze, seen from above, searching for clean water.
3. Uku Pacha — the threshold. A strobing whiteout, triggered by either collapse or finding the water.
4. Hanan Pacha — the ascent. The bird rises, whole again.
5. Hold Pacha — a held breath at the top of the ascent.
6. Restart Transition — the glyph rain, before the cycle loops back to Kay Pacha.

KEY CONCEPTS

1. State machine

The game's structure is one enum (GameState) and one switch statement in loop(). Nothing runs outside of its assigned state — this keeps the logic easy to follow even as the story gets more elaborate.

2. Analog input

The joystick reports two continuous values (0–1023), read exactly the way a potentiometer is read. Moving it up on the Y axis is what triggers a jump — a small, deliberate deviation from using the click button.

3. Memory constraints

The Arduino Uno has only 2KB of RAM, and the OLED buffer alone uses about half of that. Because of this, the maze is hand-drawn once and stored in flash memory (PROGMEM) instead of being generated by an algorithm — a real recursive maze generator would risk the little stack memory the board has left.

4. Timing without delay()

Instead of pausing the whole program between phases, the code checks millis() to know how much time has passed. This keeps the joystick responsive at every moment, even during timed sequences like the strobe or the glyph rain.

5. PROGMEM sprites

The curiingue's bitmap is stored in flash memory rather than RAM, and read back with pgm_read_byte() whenever it needs to be drawn. This is the same technique used for the maze layout, and it is what makes the sketch fit on the Uno at all.

Full source code, wiring diagrams, and updates: see the project's [GitHub repository](#).